

Resumen SQL/DES

1. Comandos de Sistema DES

/abolish: Elimina todas las tablas y vistas.

/multiline on: Permite consultas multilínea.

/duplicates off: Elimina duplicados automáticamente.

2. DDL - Definición de Datos

CREATE TABLE: Crea una nueva tabla.

```
1 create table tabla(  
2     col1 tipo primary key,  
3     col2 tipo,  
4     col3 tipo  
5 );
```

Tipos: string, int, float, etc.

PRIMARY KEY: Define clave primaria (simple o compuesta).

3. DML - Manipulación de Datos

INSERT: Inserta tuplas en una tabla.

```
1 insert into tabla  
2     values('val1','val2',123);  
3 insert into tabla(col1,col2)  
4     values('val1','val2');
```

NULL: Representa valores desconocidos/no aplicables.

4. Consultas SELECT

SELECT básico:

```
1 select columnas from tabla  
2     where condicion;
```

SELECT *: Selecciona todas las columnas.

DISTINCT: Elimina duplicados en resultados.

5. Operadores Lógicos

AND, OR, NOT: Operadores booleanos.

Comparación: =, <, >, <=, >=

IS NULL / IS NOT NULL: Comprueba valores nulos.

IN / NOT IN: Pertenencia a conjunto.

6. VISTAS (CREATE VIEW)

Las vistas son consultas almacenadas que pueden referenciarse como tablas.

```
1 create view nombre_vista as  
2     select ... from ... where ...;
```

Ventajas: Reutilización, abstracción, simplicidad.

7. Operaciones de Conjuntos

UNION: Unión de resultados (elimina duplicados).

```
1 select dni from programadores  
2 union  
3 select dni from analistas;
```

UNION DISTINCT: Unión eliminando duplicados explícitamente.

INTERSECT: Intersección de resultados.

```
1 select dni from programadores  
2 intersect  
3 select dni from analistas;
```

EXCEPT: Diferencia de conjuntos (A - B).

```
1 select dni from programadores  
2 except  
3 select dni from analistas;
```

EXCEPT ALL: Diferencia manteniendo duplicados.

8. JOIN - Combinación de Tablas

JOIN implícito (producto cartesiano con WHERE):

```
1 select * from tabla1, tabla2  
2     where tabla1.id = tabla2.id;
```

NATURAL JOIN: Join por columnas con mismo nombre.

```
1 select * from tabla1  
2     natural join tabla2;
```

JOIN explícito:

```
1 select * from tabla1  
2     join tabla2 on cond;
```

9. Funciones de Agregación

SUM(col): Suma de valores.

COUNT(*): Cuenta tuplas.

AVG(col): Media aritmética.

MAX(col): Valor máximo.

MIN(col): Valor mínimo.

```
1 select dni, sum(horas) as total  
2 from distribucion  
3 group by dni;
```

10. GROUP BY y HAVING

GROUP BY: Agrupa filas por columna(s).

```
1 select col, count(*) from tabla  
2     group by col;
```

HAVING: Filtra grupos (después de GROUP BY).

```
1 select col, count(*) from tabla  
2     group by col  
3     having count(*) > 5;
```

Diferencia: WHERE filtra antes de agrupar, HAVING después.

11. Subconsultas

Subconsulta escalar: Devuelve un único valor.

```
1 select * from tabla where col =  
2 (select max(col) from tabla);
```

Subconsulta en IN:

```
1 where dni in  
2 (select dni from analistas);
```

Subconsulta en FROM: Tabla derivada.

```
1 select * from  
2 (select dni from prog  
3 union select dni from anal);
```

12. DIVISION

Operador específico de DES para encontrar elementos que se relacionan con todos los de otro conjunto.

```
1 select dniEmp from  
2 (select codigoPr,dniEmp  
3 from distribucion)  
4 division  
5 (select codigoPr from dist  
6 where dniEmp='5');
```

Interpretación: Empleados asignados a todos los proyectos del empleado '5'.

13. Operaciones Aritméticas

Se pueden realizar operaciones en SELECT:

```
1 select codigo, horas*1.2  
2 from distribucion;
```

Operadores: +, -, *, /

14. Alias y RENAME

AS: Define alias para columnas/tablas.

```
1 select dni as identificador,  
2 sum(horas) as total  
3 from distribucion;
```

RENAME (sintaxis DES):

```
1 rename (dniEmp as dni)  
2 (distribucion);
```

15. Consultas Complejas

Combinando múltiples operaciones:

```
1 create view vista_compleja as  
2 (select dni,nombre,codigoPr  
3 from (select * from prog  
4 union select * from anal)  
5 join dist on dni = dniEmp)  
6 union  
7 (select dni,nombre,codigo  
8 from (select * from prog  
9 union select * from anal)  
10 join proy on dni = dniDir);
```

16. Buenas Prácticas

- Usar vistas para consultas complejas reutilizables
- Aplicar filtros WHERE antes de JOIN cuando sea posible
- Usar DISTINCT solo cuando sea necesario (impacto en rendimiento)
- Nombrar columnas con alias descriptivos
- Dividir consultas complejas en vistas intermedias
- Considerar NULL en comparaciones (NULL != NULL)

17. Orden de Ejecución SQL

Orden lógico de evaluación:

1. FROM (tablas/joins)
2. WHERE (filtrado de filas)
3. GROUP BY (agrupación)
4. HAVING (filtrado de grupos)
5. SELECT (proyección)
6. UNION/INTERSECT/EXCEPT
7. ORDER BY (no usado en ejemplos)

18. Ejemplos Prácticos

Empleados sin teléfono:

```
1 select dni,nombre from prog  
2 where telefono is null  
3 union  
4 select dni,nombre from anal  
5 where telefono is null;
```

Suma de horas por empleado:

```
1 select dni, sum(horas) as total  
2 from (select dni from prog  
3 union select dni from anal)  
4 join dist on dni = dniEmp  
5 group by dni;
```

Proyectos sin analistas:

```
1 select codigo from proyectos  
2 except  
3 select distinct codigo from  
4 (select codigoPr as codigo,  
5 dniEmp from dist)  
6 join analistas on dniEmp = dni;
```

19. WITH - Expresiones de Tabla Comunes (CTE)

WITH ... AS permite definir tablas temporales nombradas (CTE) que simplifican consultas complejas y mejoran legibilidad. Se pueden encadenar varias CTE y usar WITH RECURSIVE para recursión.

Sintaxis básica:

```
1 WITH nombre_cte AS (  
2 select ...  
3 )  
4 SELECT ... FROM nombre_cte;
```

Ejemplo 1: sumar horas por empleado y filtrar

```
1 WITH total_horas AS (  
2 SELECT dniEmp, SUM(horas) AS total  
3 FROM distribución  
4 GROUP BY dniEmp  
5 )  
6 SELECT dniEmp, total  
7 FROM total_horas  
8 WHERE total > 20;
```

Ejemplo 2: equivalente a DIVISION (empleados que hacen todos los proyectos de 'Evaristo')

```
1 WITH proyectos_eva AS (  
2     SELECT códigoPr  
3     FROM distribución  
4     WHERE dniEmp = (SELECT dni FROM analistas WHERE  
5         nombre = 'Evaristo')  
6 ),  
7 empleados_conjunto AS (  
8     SELECT dniEmp  
9     FROM distribución  
10    WHERE códigoPr IN (SELECT códigoPr FROM  
11        proyectos_eva)  
12    GROUP BY dniEmp  
13    HAVING COUNT(DISTINCT códigoPr) = (SELECT COUNT(*)  
14        FROM proyectos_eva)  
15 )  
16 SELECT dniEmp FROM empleados_conjunto;
```

Ejemplo 3: CTE recursiva (generar números)

```
1 WITH RECURSIVE nums(n) AS (  
2     SELECT 1  
3     UNION ALL  
4     SELECT n+1 FROM nums WHERE n < 5  
5 )  
6 SELECT * FROM nums;
```

Notas:

- MySQL soporta WITH desde la versión 8.0; WITH RECURSIVE para recursión.
- Las CTE mejoran legibilidad y evitan subconsultas repetidas.
- Usar CTE en vez de vistas si la tabla temporal solo se usa dentro de la consulta.